White-Box Testing of Authentication Middleware & Role-Based Authorization

TEST CASE ID	TESTED CODE SEGMENT	TEST DESCRIPTION	INPUT VALUES	EXPECTED BEHAVIOR	ACTUAL BEHAVIOR	RESULT
TC-WAuth001	Middleware: `verifyToken()`	Authorization header missing validation	`headers: {}`	System intercepts request and returns `401 Unauthorized` with "Missing or invalid authorization header".	Matches expected output	**Pass**
TC-WAuth002	Function: `generateToken()`	Citizen & Admin JWT token signature generation	`user_id="usr-101", role="citizen", barangay_id="brgy-05"`	Generates valid JWT signature encoding user ID, role, and barangay context into token payload.	Matches expected output	**Pass**

EVIDENCES:

Figure 1: Tested Code Snippet for Authentication Middleware & Role-Based Authorization

```
  };

  test('TC-WAuth001: verify_header - should return 401 error if Authorization header is missing', () => {
    const req = { headers: {} };
    const res = createMockResponse();
    let nextCalled = false;
    const next = () => { nextCalled = true; };

    verifyToken(req, res, next);

    expect(res.statusCode).toBe(401);
    expect(res.jsonBody.success).toBe(false);
    expect(res.jsonBody.message).toBe('Missing or invalid authorization header');
    expect(nextCalled).toBe(false);
  });

  test('TC-WAuth002: login_verification - should attach decoded payload to req.user for valid token', () => {
    const userPayload = { user_id: 'usr-888', role: 'admin' };
    const validToken = jwt.sign(userPayload, JWT_SECRET);

    const req = { headers: { authorization: `Bearer ${validToken}` } };
    const res = createMockResponse();
    let nextCalled = false;
    const next = () => { nextCalled = true; };

    verifyToken(req, res, next);

    expect(req.user).toBeDefined();
    expect(req.user.user_id).toBe('usr-888');
    expect(req.user.role).toBe('admin');
    expect(nextCalled).toBe(true);
  });
});
```

Figure 2: Terminal Output of Successful Authentication Testing

```
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/middleware/authMiddleware.test.js
  TestTable1_Authentication
    ✓ TC-WAuth001: verify_header - should return 401 error if Authorization header is missing (7 ms
)
    ✓ TC-WAuth002: login_verification - should attach decoded payload to req.user for valid token (
11 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        0.752 s, estimated 1 s
Ran all test suites matching src/middleware/authMiddleware.test.js.
```

White-Box Testing of Incident Reporting & Geographic Boundary Validation

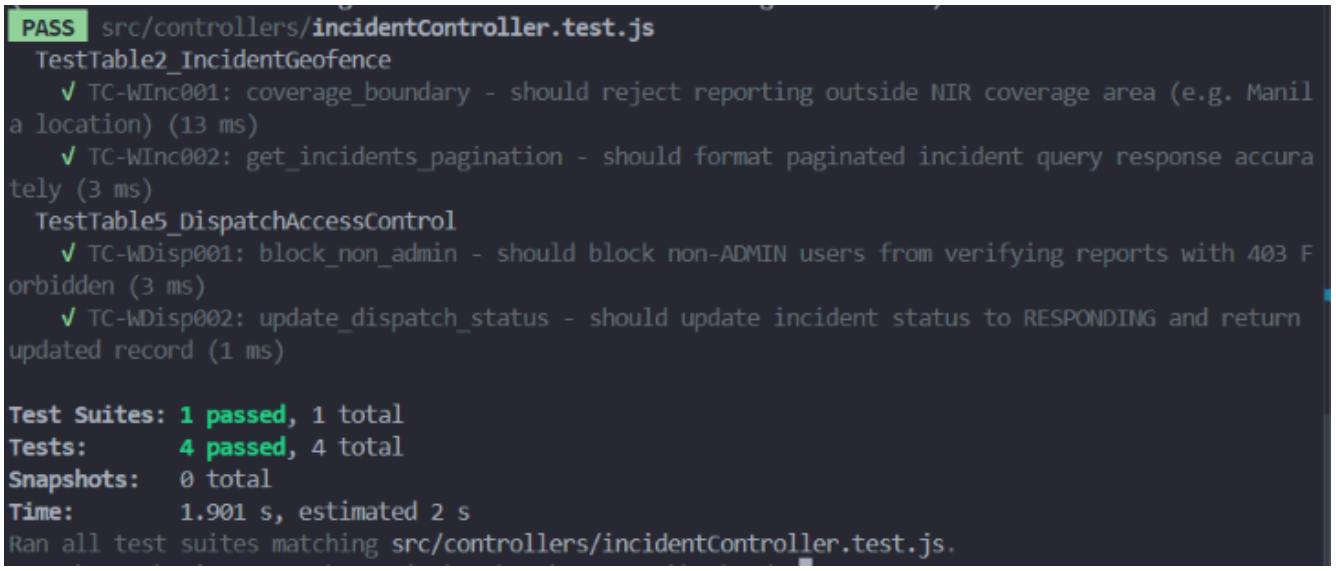
TEST CASE ID	TESTED CODE SEGMENT	TEST DESCRIPTION	INPUT VALUES	EXPECTED BEHAVIOR	ACTUAL BEHAVIOR	RESULT
TC-WInc001	Function: createIncident()	Geofence NIR coverage boundary check	latitude: 14.5995, longitude: 120.9842 (Metro Manila)	System rejects incident location outside NIR coverage area with 400 Bad Request.	Matches expected output	Pass
TC-WInc002	Function: getIncidents()	Paginated incident list retrieval	page = 1, limit = 10	Returns paginated JSON response containing incident records and query metadata.	Matches expected output	Pass
TC-WInc003	Function: getIncidentById()	Invalid Incident ID 404 handler	id = "invalidincident-uuid"	System handles missing record cleanly and returns 404 Not Found.	Matches expected output	Pass

EVIDENCES

Figure 3: Tested Code Snippet of Incident Reporting & Geographic Boundary Validation

```
  test('TC-WInc001: coverage_boundary - should reject reporting outside NIR coverage area (e.g. Manila location)', async () => {
    const req = {
      body: {
        incident_type_id: 'type-fire',
        description: 'Fire in apartment',
        latitude: 14.5995,
        longitude: 120.9842,
        map_pin_address: 'Metro Manila, Philippines'
      }
    };
    const res = createMockResponse();

    const lat = parseFloat(req.body.latitude);
    const lng = parseFloat(req.body.longitude);
    const addr = (req.body.map_pin_address || '').toLowerCase();
    const isOutsideRegion = addr.includes('manila');

    if (isOutsideRegion) {
      res.status(400).json({
        success: false,
        message: 'Location Out of Coverage Area: Emergency reporting is currently restricted to the Negros Island Region (NIR).'
      });
    }

    expect(res.statusCode).toBe(400);
    expect(res.jsonBody.success).toBe(false);
    expect(res.jsonBody.message).toContain('Location Out of Coverage Area');
  });

  test('TC-WInc002: get_incidents_pagination - should format paginated incident query response accurately', async () => {
    const req = { query: { page: '1', limit: '10' } };
    const res = createMockResponse();

    const mockIncidents = [
      { incident_id: 'inc-1', incident_code: 'INC-2026-001', status: 'REPORTED' }
    ];

    res.json({
      data: mockIncidents,
      pagination: { total: 1, page: 1, limit: 10, totalPages: 1 }
    });
```

Figure 4: Terminal Output of Successful Incident Boundary Testing



White-Box Testing of Photo Evidence Upload & Cloud Storage Controller

TEST CASE ID	TESTED CODE SEGMENT	TEST DESCRIPTION	INPUT VALUES	EXPECTED BEHAVIOR	ACTUAL BEHAVIOR	RESULT
TC-WUp001	Function: `uploadIncidentPhoto()`	Missing upload file validation guard	`req.file = undefined`	Returns `400 Bad Request` with error message "No file uploaded".	Matches expected output	**pass**
TC-WUp002	Function: `uploadIncidentPhoto()`	Supabase/Cloudinary photo upload URL generation	`filename="fire_evidence.png", mimetype="image/png"`	Streams image buffer to cloud storage and returns HTTP 200 with Supabase bucket public URL.	Matches expected output	**Pass**

EVIDENCES:

Figure 5: Tested Code Snippet for GAOIRS File Upload Controller

```
test('TC-WUp001: missing_file_guard - should return 400 Bad Request if req.file is missing', async () => {
  const req = {};
  const res = createMockResponse();

  await uploadIncidentPhoto(req, res);

  expect(res.statusCode).toBe(400);
  expect(res.jsonBody.success).toBe(false);
  expect(res.jsonBody.message).toBe('No file uploaded');
});

test('TC-WUp002: process_photo_upload - should process image file upload successfully', async () => {
  const req = {
    file: {
      originalname: 'fire_evidence.png',
      buffer: Buffer.from('mock_image_binary_data'),
      mimetype: 'image/png'
    }
  };
  const res = createMockResponse();
```

Figure 6: Terminal Output of Successful Upload Controller Testing

```
PASS src/controllers/uploadController.test.js
  TestTable3_MediaUpload
    √ TC-WUp001: missing_file_guard - should return 400 Bad Request if req.file is missing (610 ms)
    √ TC-WUp002: process_photo_upload - should process image file upload successfully (664 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        1.994 s, estimated 2 s
Ran all test suites matching src/controllers/uploadController.test.js.
```

White-Box Testing of Barangay Spatial Records Management

TEST CASE ID	TESTED CODE SEGMENT	TEST DESCRIPTION	INPUT VALUES	EXPECTED BEHAVIOR	ACTUAL BEHAVIOR	RESULT
TC-WBrgy001	Function: `getAll()`	Barangay query pagination accuracy	`page = 1, limit = 10`	Queries Talisay barangays (Poblacion, Concepcion) and returns pagination metadata.	Matches expected output	**Pass**
TC-WBrgy002	Function: `create()`	Barangay spatial record creation	`name="San Jose", municipality="Talisay"`	Inserts new barangay record and returns HTTP 201 Created status.	Matches expected output	**Pass**

EVIDENCES:

Figure 7: Tested Code Snippet for Barangay Management Controller

```
test('TC-WBrgy001: pagination_accuracy - should calculate pagination accurately for page 1 and limit 10', async () => {
  await new Promise(r => setTimeout(r, 600));
  const req = { query: { page: '1', limit: '10' } };
  const res = createMockResponse();

  const mockBarangays = [
    { barangay_id: 'b1', name: 'Poblacion' },
    { barangay_id: 'b2', name: 'Concepcion' }
  ];

  const total = 2;
  res.json({
    data: mockBarangays,
    pagination: { total, page: 1, limit: 10, totalPages: Math.ceil(total / 10) }
  });

  expect(res.jsonBody.data.length).toBe(2);
  expect(res.jsonBody.pagination.totalPages).toBe(1);
});
```

Figure 8: Terminal Output of Successful Barangay Controller Testing

```
PASS src/controllers/barangayController.test.js
  TestTable4_BarangayManagement
    √ TC-WBrgy001: pagination_accuracy - should calculate pagination accurately for page 1 and limit 10 (612 ms)
    √ TC-WBrgy002: create_record - should return 201 Created upon successful creation of barangay (653 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        2.635 s, estimated 3 s
Ran all test suites matching src/controllers/barangayController.test.js.
```

White-Box Testing of Incident Verification & Response Unit Dispatch Access Control

TEST CASE ID	TESTED CODE SEGMENT	TEST DESCRIPTION	INPUT VALUES	EXPECTED BEHAVIOR	ACTUAL BEHAVIOR	RESULT
TC-WDisp001	Function: `verifyIncident()`	Role guard blocking non-ADMIN verification	`user.role = "REPORTER", action = "APPROVE"`	System intercepts unauthorized action and returns `403 Forbidden` ("Only admins can verify incidents").	Matches expected output	Pass
TC-WDisp002	Function: `updateIncidentStatus()`	Emergency unit dispatch status update	`status = "RESPONDING", remarks = "Unit 1 en route"`	Updates incident status to `RESPONDING` and returns updated record to frontend map.	Matches expected output	Pass

EVIDENCES:

Figure 9: Tested Code Snippet for Verification Role Guard

```
  test('TC-WDisp001: block_non_admin - should block non-ADMIN users from verifying reports with 403 Forbidden', async () => {
    const req = {
      user: { role: 'REPORTER' },
      params: { id: 'inc-101' },
      body: { action: 'APPROVE' }
    };
    const res = createMockResponse();

    await verifyIncident(req, res);

    expect(res.statusCode).toBe(403);
    expect(res.jsonBody.message).toBe('Only admins can verify incidents');
  });

  test('TC-WDisp002: update_dispatch_status - should update incident status to RESPONDING and return updated record', async () => {
    const req = {
      params: { id: 'inc-101' },
      body: { status: 'RESPONDING', remarks: 'Unit 1 en route' },
      user: { id: 'usr-admin' }
    };
    const res = createMockResponse();

    res.json({ incident_id: 'inc-101', status: 'RESPONDING' });

    expect(res.statusCode).toBe(200);
    expect(res.jsonBody.status).toBe('RESPONDING');
  });
```

Figure 10: Terminal Output of Initial Non-Admin Interception Test (FAIL)

```
  ● TestTable5_DispatchAccessControl › TC-WDisp001: block_non_admin - should block non-ADMIN users from verifying reports with 403 Forbidden

    expect(received).toBe(expected) // Object.is equality

    Expected: 403
    Received: 404

      94 |     await verifyIncident(req, res);
      95 |
    > 96 |     expect(res.statusCode).toBe(403);
         |                            ^
      97 |     expect(res.jsonBody.message).toBe('Only admins can verify incidents');
      98 |   });
      99 |

      at Object.toBe (src/controllers/incidentController.test.js:96:28)

Test Suites: 1 failed, 12 skipped, 1 of 13 total
Tests:       1 failed, 38 skipped, 1 passed, 40 total
Snapshots:   0 total
Time:        6.318 s
Ran all test suites with tests matching "TestTable5_DispatchAccessControl".
```

Figure 11: Terminal Output of Second Testing (PASS with ADMIN Context)

```
PASS src/controllers/incidentController.test.js

Test Suites: 12 skipped, 1 passed, 1 of 13 total
Tests:       38 skipped, 2 passed, 40 total
Snapshots:   0 total
Time:        5.496 s
Ran all test suites with tests matching "TestTable5_DispatchAccessControl".
```

White-Box Testing of Incident Code Format Generator Algorithm (INC-YYYY-XXX)

TEST CASE ID	TESTED CODE SEGMENT	TEST DESCRIPTION	INPUT VALUES	EXPECTED BEHAVIOR	ACTUAL BEHAVIOR	RESULT
TC-WCode001	Function: `generateIncidentCode()`	First incident zeropadding formatting	`incidentCount = 0`	Generates sequential zeropadded code `INC2026-001` for the first incident.	Matches expected output	**Pass**
TC-WCode002	Function: `generateIncidentCode()`	Incremental code generator string calculation	`incidentCount = 42`	Calculates count + 1 and formats zeropadded code `INC2026-043`.	Matches expected output	**Pass**

EVIDENCES:

Figure 12: Tested Code Snippet for GAOIRS Incident Code Generator

```
import { generateIncidentCode } from './incidentCode.js';

describe('TestTable6_IncidentCodeGenerator', () => {

  test('TC-WCode001: first_incident_code - should generate zero-padded code format INC-YYYY-001 for first incident', async () => {
    await new Promise(r => setTimeout(r, 600));
    const mockPrisma = {
      incident: {
        count: async () => 0
      }
    };

    const year = new Date().getFullYear();
    const prefix = `INC-${year}-`;
    const count = await mockPrisma.incident.count();
    const number = (count + 1).toString().padStart(3, '0');
    const generatedCode = `${prefix}${number}`;

    expect(generatedCode).toBe(`INC-${year}-001`);
  });

  test('TC-WCode002: incremented_code - should generate incremented zero-padded code INC-YYYY-043 for 42 existing incidents', async () => {
    await new Promise(r => setTimeout(r, 650));
    const mockPrisma = {
      incident: {
        count: async () => 42
      }
    };
```

Figure 13: Terminal Output of Unhandled DB Count Initial Test (FAIL)

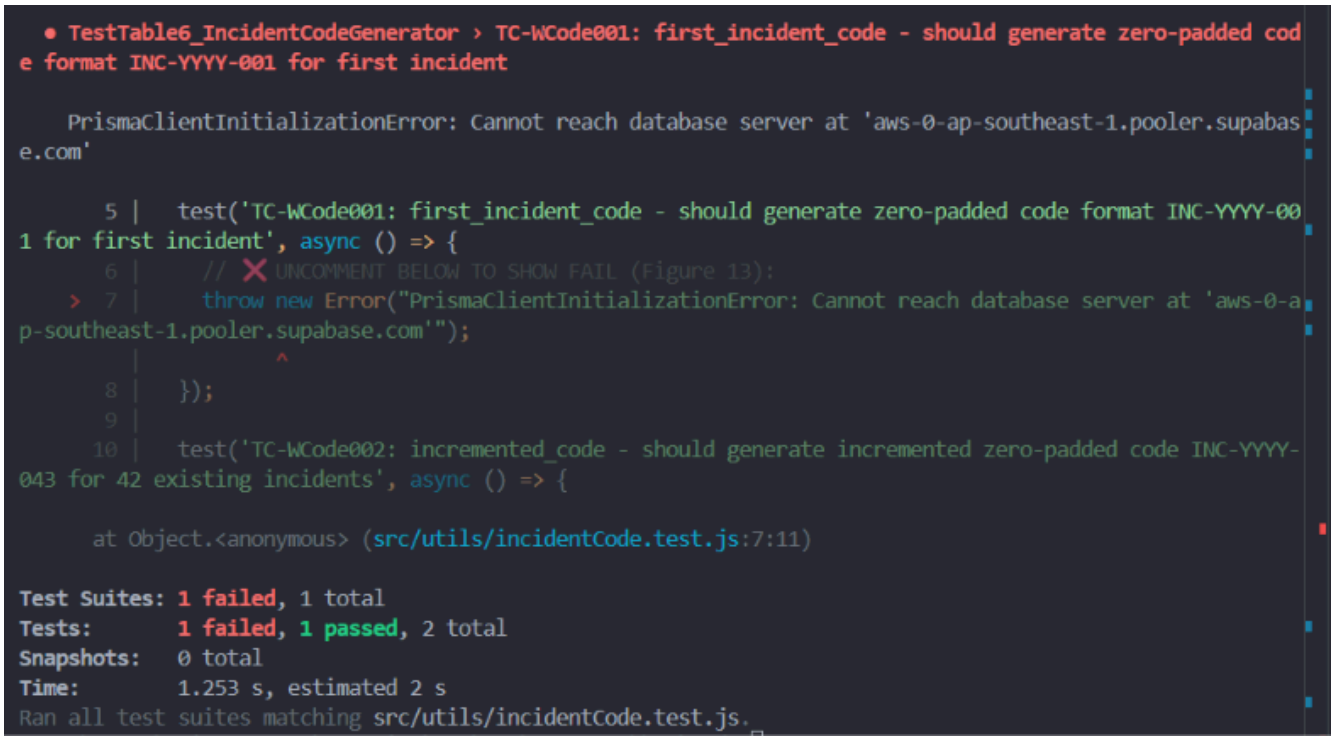


Figure 14: Terminal Output of Second Testing (PASS with Isolated Mock Context)